VS RAG Architecture

Purpose

The RAG pipeline predicts value streams for a new idea card or Jira ticket by combining:

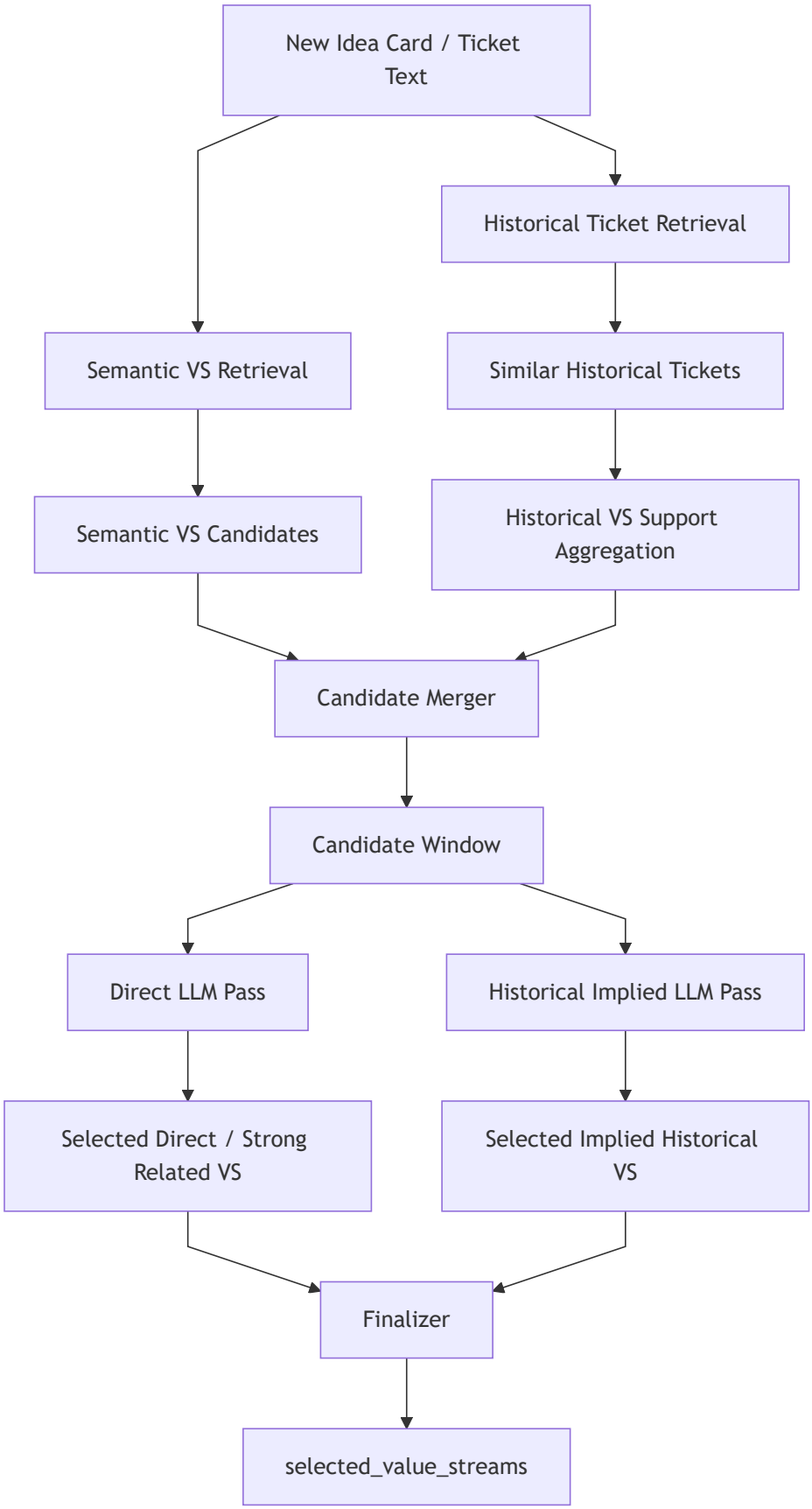
```
1. current-card semantic evidence
2. similar historical ticket evidence
3. LLM adjudication
```

Core principle:

```
Retrieval finds evidence.
Merge organizes evidence.
LLM decides.
Finalizer cleans and dedupes only.
```

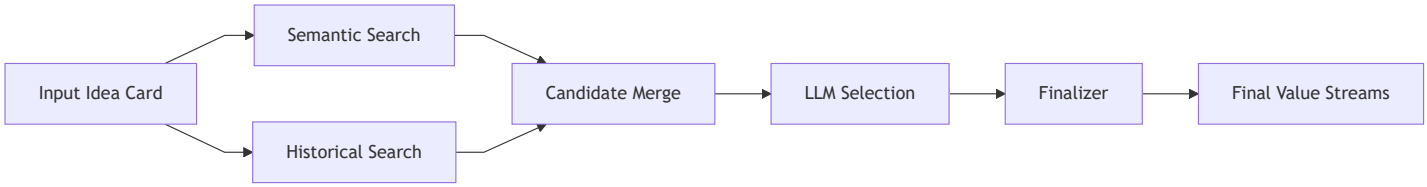
No auto-select.
No hidden rescue.
No threshold-based truth decision.

High-Level Flow



Condensed Overview Diagram

This is a short, UI-style overview of the same flow.



Condensed Flow Summary

```
Input idea card
→ semantic retrieval
→ historical retrieval
→ merge candidates and evidence
→ LLM selects direct and implied value streams
→ finalizer sanitizes and dedupes
→ final selected_value_streams
```

Main Files

```
src/vs_app/modules/rag/augmentation/candidate_merger.py
src/vs_app/modules/rag/augmentation/finalizer.py
src/vs_app/modules/rag/augmentation/prompt_context.py
src/vs_app/modules/rag/ranking/reranker.py
src/vs_app/modules/prompts/loader.py
```

Key Design Change

Old behavior:

```
candidate_merger.py
  = merge + score formulas + auto-select + auto-drop + quota math

finalizer.py
  = LLM + rescue engine + threshold filtering + dedupe
```

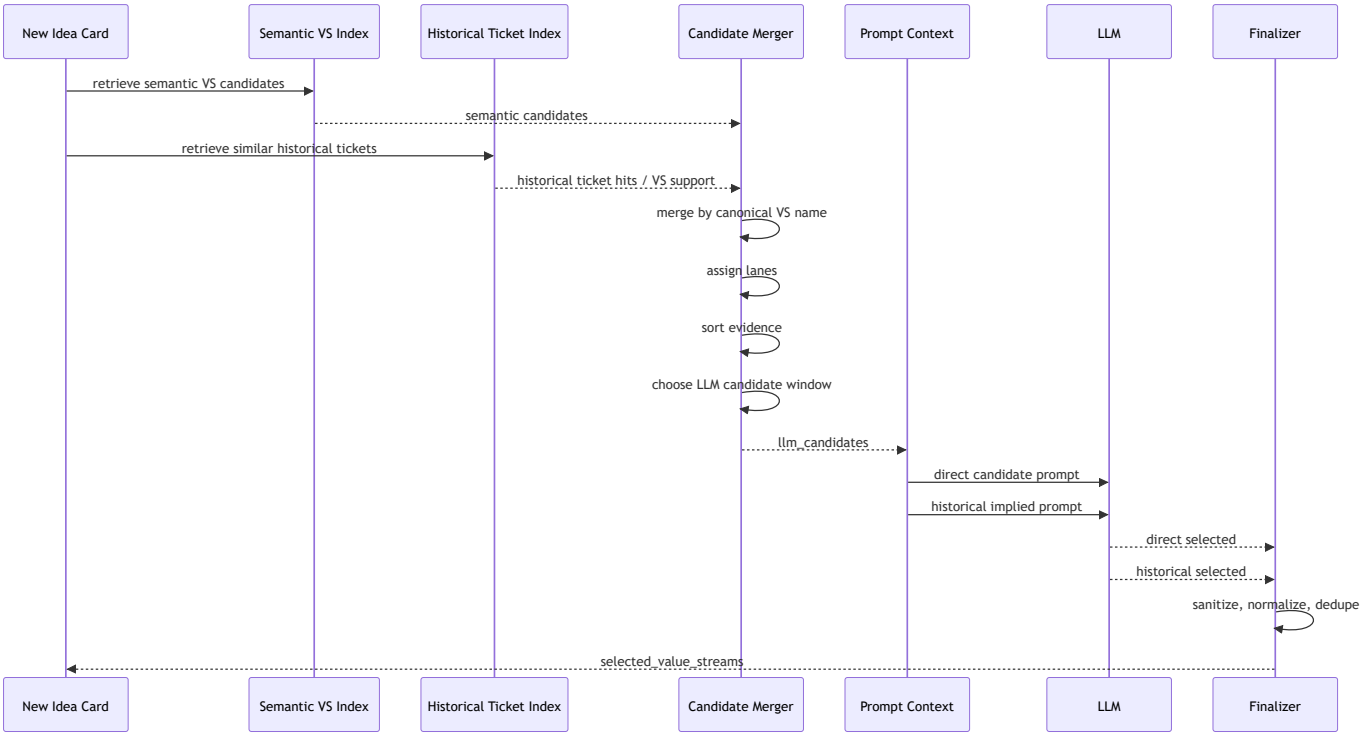
New behavior:

```
candidate_merger.py
  = evidence merge + lane assignment + prompt-window selection

finalizer.py
  = run LLM passes + sanitize + dedupe

LLM
  = only component that decides selected value streams
```

End-to-End Sequence



Candidate Sources

1. Semantic candidates

Semantic candidates come from matching the new idea card against the value-stream index.

Example:

```
{
  "entity_id": "VS-042",
  "entity_name": "Manage Invoice and Payment Receipt",
  "description": "Manage invoice capture, validation, payment posting, and receipt handlin
g.",
  "semantic_score": 1.31
}
```

Semantic score means:

```
How closely the current idea-card text matches this value stream.
```

Semantic score is evidence only. It does not auto-select the value stream.

2. Historical candidates

Historical candidates come from similar old tickets.

Each historical ticket already has:

```
direct_vs_names
implied_vs_names
summary
similarity score
```

Historical support is aggregated per value stream.

Example:

```
{
  "entity_name": "Issue Payment",
  "supporting_ticket_count": 2,
  "direct_count": 1,
  "implied_count": 1,
  "best_support_score": 0.86,
  "avg_support_score": 0.82,
  "weighted_support": 1.6,
  "supporting_ticket_ids": ["IDMT-123", "IDMT-456"],
  "historical_reasons": [
    "[IDMT-123 / direct] Prior similar ticket directly involved issuing payment.",
    "[IDMT-456 / implied] Similar funding workflow implied payment operations."
  ]
}
```

Historical Evidence Terms

Field	Meaning
`supporting_ticket_count`	Number of unique retrieved historical tickets supporting this VS
`support_count`	Backward-compatible alias for `supporting_ticket_count`
`direct_count`	Count of supporting historical tickets where this VS was direct
`implied_count`	Count of supporting historical tickets where this VS was implied
`best_support_score`	Highest similarity score among supporting historical tickets
`avg_support_score`	Average similarity score among supporting historical tickets
`weighted_support`	Weighted historical support based on similarity strength
`supporting_ticket_ids`	Historical ticket IDs used as evidence
`historical_reasons`	Short analog reasons from historical records

Important:

```
Use query-local historical evidence only.
Do not use global corpus frequency as a truth signal.
```

Historical Support Weighting

Historical similarity is used to weight evidence for sorting, not to auto-select.

Recommended weighting:

```
def historical_support_weight(score: float) -> float:
    if score >= 0.80:
        return 1.0
    if score >= 0.70:
        return 0.6
    if score >= 0.60:
        return 0.3
    return 0.0
```

Example:

```
Issue Payment:
  H1 similarity 0.86 → weight 1.0
  H2 similarity 0.79 → weight 0.6
  weighted_support = 1.6

Generic Candidate:
  H1 similarity 0.61 → weight 0.3
  H2 similarity 0.60 → weight 0.3
  weighted_support = 0.6
```

This prevents raw count from dominating stronger semantic matches.

Candidate Merge

File:

```
src/vs_app/modules/rag/augmentation/candidate_merger.py
```

Merge key:

```
canonical value-stream name
```

The merger combines semantic and historical evidence into one candidate row.

Example merged candidate:

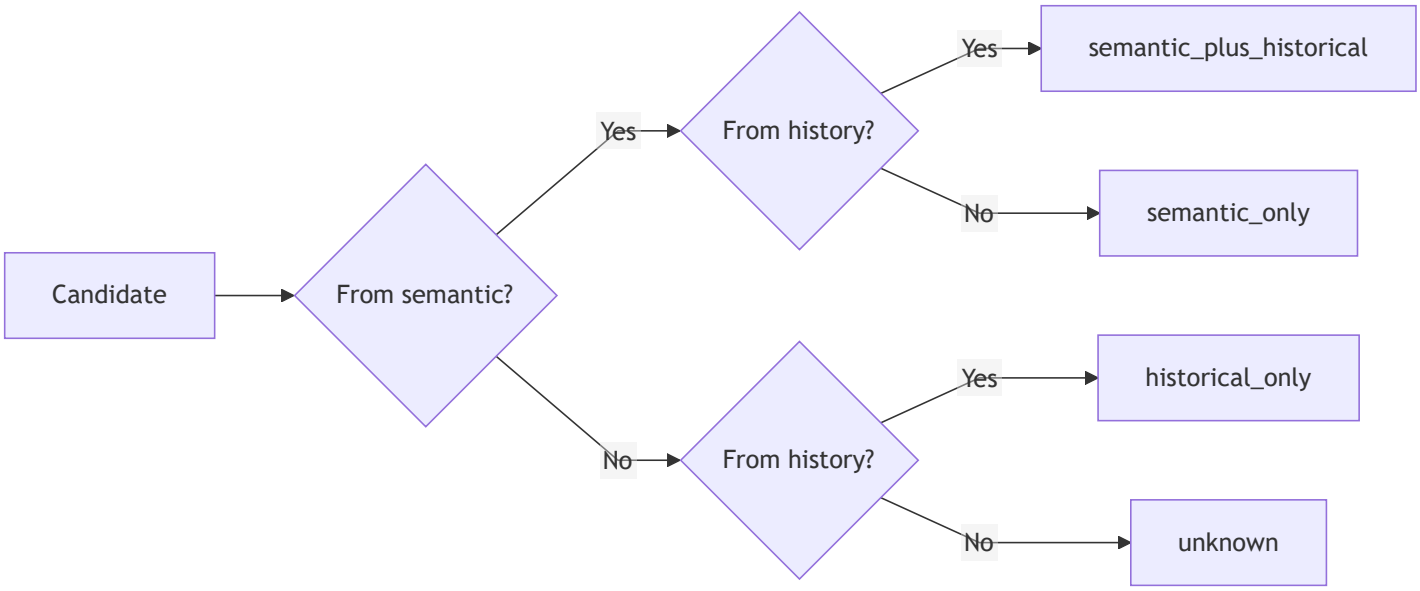

```
{
  "entity_id": "VS-042",
  "entity_name": "Manage Invoice and Payment Receipt",
  "lane": "semantic_plus_historical",
  "bucket": "semantic_plus_historical",
  "candidate_lane": "semantic_plus_historical",

  "from_semantic": true,
  "semantic_score": 1.31,
  "semantic_rank": 3,

  "from_historical": true,
  "supporting_ticket_count": 3,
  "support_count": 3,
  "direct_count": 1,
  "implied_count": 2,
  "best_support_score": 0.84,
  "avg_support_score": 0.78,
  "weighted_support": 1.6,
  "supporting_ticket_ids": ["IDMT-1", "IDMT-2", "IDMT-3"],
  "historical_reasons": []
}
```


Candidate Lanes

The system uses exactly three lanes.



`semantic_plus_historical`

Found by current-card semantic retrieval and supported by historical matches.

This is usually the strongest evidence lane because both current text and past analogs agree.

`semantic_only`

Found by current-card semantic retrieval only.

This is important for new idea cards where there may be no good historical analog.

`historical_only`

Found only through similar historical tickets.

This is where implied value streams usually appear.

Candidate Window Policy

Current default:

```
CandidateWindowPolicy(
    max_semantic_plus_historical=8,
    max_semantic_only=12,
    max_historical_only=10,
    max_supporting_tickets_per_candidate=3,
)
```

Total LLM candidate window:

```
8 + 12 + 10 = 30 candidates
```

These are prompt-size controls, not truth thresholds.

Candidate Sorting

Sorting decides what fits into the LLM candidate window.

It does not decide final truth.

`semantic_plus_historical` sorting

Priority:

```
1. semantic_score desc
2. best_support_score desc
3. weighted_support desc
4. supporting_ticket_count desc
5. entity_name asc
```

Reason:

```
Prefer candidates that match the current idea card and have strong historical analogs.
```

`semantic_only` sorting

Priority:

```
1. semantic_score desc
2. entity_name asc
```

Reason:

```
For candidates with no history, current-card semantic relevance is the main evidence.
```

`historical_only` sorting

Priority:

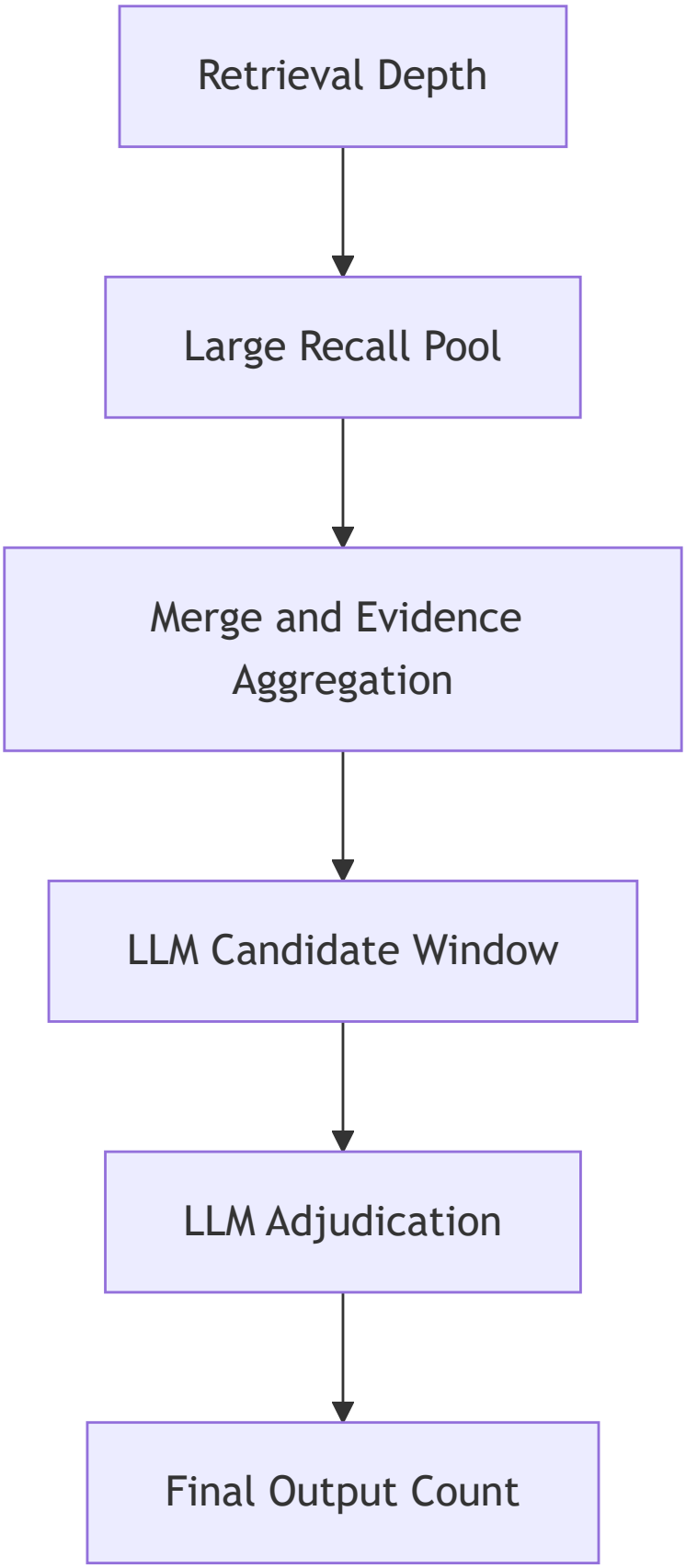
```
1. best_support_score desc
2. weighted_support desc
3. direct_count desc
4. supporting_ticket_count desc
5. implied_count desc
6. avg_support_score desc
7. entity_name asc
```

Reason:

```
One or two very close historical analogs can be more valuable than many weak generic matche
s.
```

Retrieval vs LLM Window vs Final Output

These must stay separate.



Recommended operational defaults:

```
semantic_fetch_k = 40
historical_ticket_fetch_k = 35-40
llm_candidate_window = 30
final_output_count = user slider
```

Important:

```
The user slider should control final_output_count only.
It should not reduce retrieval depth or candidate merge depth.
```

Example:

```
User asks for top 8 final VS.

System still:
- retrieves 40 semantic candidates
- retrieves 35-40 historical tickets
- sends up to 30 candidates to LLM
- asks LLM to return up to 8 final value streams
```

Direct LLM Pass

Input lanes:

```
semantic_plus_historical
semantic_only
```

Purpose:

```
Select value streams directly or strongly supported by the current idea card.
```

The prompt may include semantic and historical evidence, but it should not tell the LLM to select because a score is high.

The LLM decides whether the value stream is actually relevant.

Historical Implied LLM Pass

Input lane:

```
historical_only
```

Purpose:

```
Evaluate whether a pattern from similar historical tickets transfers to the new idea card.
```

This pass handles candidates that do not appear semantically obvious in the current idea card, but are suggested by past analogs.

Prompt structure:

```
HISTORICAL TICKET CONTEXT
[H1] IDMT-18422
Similarity: 0.84
Summary: ...
Direct value streams: ...
Implied value streams: ...

HISTORICAL CANDIDATES TO ADJUDICATE
1. Issue Payment
Historical support: 2 unique tickets
Best historical similarity: 0.86
Analog evidence:
  - [H1 / direct] ...
  - [H2 / implied] ...

Question:
Does this historical pattern transfer to the new idea card?
```

Prompt Context

File:

```
src/vs_app/modules/rag/augmentation/prompt_context.py
```

Candidate blocks include:

```
Lane
Description
Semantic score
Historical support
Best historical similarity
Average historical similarity
Weighted historical support
Supporting tickets
Analog evidence
```

Candidate blocks should not expose old formula fields as decision logic:

```
ranking_score
historical_strength
```

Those fields are not part of the new architecture.

Finalizer

File:

```
src/vs_app/modules/rag/augmentation/finalizer.py
```

The finalizer now does:

```
1. split candidates into direct and historical passes
2. run both LLM passes
3. sanitize selected outputs against candidate list
4. rewrite score-heavy reasons if needed
5. merge and dedupe selections
6. return selected_value_streams
```

Finalizer does not:

```
- rescue candidates
- auto-select candidates
- add candidates LLM rejected
- drop LLM-selected candidates using historical thresholds
```

Compatibility keys may still be returned as empty lists:

```
rescued_confirmed_merged_value_streams
rescued_historical_gap_fill_value_streams
dropped_historical_gap_fill_value_streams
```

These are legacy/debug compatibility outputs only.

Final Output Shape

Typical output:

```
{
  "selected_value_streams": [
    {
      "entity_id": "VS-042",
      "entity_name": "Manage Invoice and Payment Receipt",
      "confidence": 0.82,
      "reason": "Selected because the idea card aligns with invoice/payment operations.",
      "supporting_ticket_ids": ["IDMT-123"]
    }
  ],
  "llm_selected_value_streams": [],
  "raw_response": {
    "selected_value_streams": [],
    "direct_pass": {},
    "historical_gap_pass": {},
    "auto_selected_ignored": []
  },
  "candidates_used": []
}
```

What Was Removed Conceptually

The new architecture removes active reliance on:

```
auto_selected_value_streams
_should_auto_include
_should_auto_include_merged
historical_strength formula
ranking_score formula
dropped_before_llm as truth
confirmed_direct / historical_recall / semantic_direct old lanes
rescue_confirmed_merged
rescue_historical_gap_fill
gap-fill threshold rescue
```

Why No Rescue?

Rescue means code adds candidates after the LLM did not select them.

That creates hidden decision logic.

Bad:

```
LLM rejects candidate
→ code rescues it because support_count or best_score is high
```

Good:

```
Candidate gets into the LLM window
→ LLM sees all evidence
→ LLM decides
→ finalizer only dedupes/sanitizes
```

If a candidate is strong but unselected, keep it in debug/rejected candidates, not in `selected_value_streams`.

Why Not Raw Historical Count?

Raw count can create popularity bias.

Example:

```
Manage Member Care appears in many weak historical tickets.
Issue Payment appears in two very close historical tickets.
```

The second may be more relevant.

That is why historical-only sorting prioritizes:

```
best_support_score
weighted_support
direct_count
supporting_ticket_count
avg_support_score
```

not just raw count.

Default Limits Summary

```
semantic_fetch_k:                  40
historical_ticket_fetch_k:          35-40
max_semantic_plus_historical:        8
max_semantic_only:                  12
max_historical_only:                10
max_llm_candidates:                 30
max_supporting_tickets_per_candidate: 3
max_historical_ticket_context:      10
```

RAG Architecture Summary

```
New idea card
→ semantic value-stream retrieval
→ historical ticket retrieval
→ query-local historical VS aggregation
→ merge by VS name
→ assign lane
→ sort within lanes
→ send up to 30 candidates to LLM
→ direct LLM pass
→ historical implied LLM pass
→ sanitize/dedupe
→ selected_value_streams
```

This keeps recall high without letting thresholds or rescue logic decide truth.